Module 1
Foundations of Generative AI and Prompt Engineering
Module 2
Introduction to LangChain in GenAI Applications
LangChain Core Components and Advanced Features
Recap: Introduction to LangChain
Reading • 10 min
Lab: LangChain Models
Graded Lab • 30 min
LangChain Chains and Agents for Building Applications
Video • 10 min
Tutorial: LangChain LCEL (Tuning Method)
Video • 10 min
Lab: Build, Evaluate & Apps (Sequential, LCEL with Memory)
Graded Lab • 1 hr 30 min
Practice Quiz: LangChain Core Components and Advanced Features
Graded Assignment • 3 Qu... 45m
Module Summary and Evaluation
Summary and Highlights: Introduction to LangChain for GenAI Applications
Reading • 10 min
Graded Quiz: Introduction to LangChain for GenAI Applications
Graded Assignment • 4 Qu... 15m
Module 3
Build Interactive Web Applications with LangChain
Application Development Workflow and Scenarios in Industry
Choose the Right AI Model for Your Use Case
Reading • 10 min
Hands-On: Building Applications with Streamlit
Video • 10 min
Streamlit Starter Code
Video • 10 min
Streamlit Reading: From Prototypes to Web Development in Python
Reading • 10 min
Optional Reading: Python with Flask for Large Scale Projects
Reading • 10 min
Final Week: Web Development Using Flask
Reading • 10 min
Reading and Discussion: Choosing the Right Model for Your Application
Graded Assignment • 3 Qu...
Practice Quiz: Application Development Workflow and Scenarios in Industry
Graded Assignment • 3 min
Summary and Evaluation
Summary and Highlights: Build Interactive AI Applications with LangChain
Reading • 10 min
Final Week: Build Generative AI Application with LangChain
Graded Quiz • 1 hr
Graded Quiz: Build, Evaluate & Deploy AI Applications with LangChain
Graded Assignment • 12 min
Course Wrap Up
Course Wrap Up
Video • 10 min
Congratulations and Next Steps
Reading • 10 min
Team Collaboration Agreement
Reading • 10 min

(Optional) Reading: Python with Flask for Large Scale Projects

Introduction

Python with Flask is a lightweight and flexible web application framework. It's known for its simplicity, its modularity, and its ease of use. Flask is a great choice for developing small to medium-sized web applications, including web applications, and many Flask extensions have been developed to add more functionality to Flask, making it a powerful framework for building a wide range of web applications. Flask's modular design allows developers to easily integrate additional libraries and tools as needed, making it a versatile framework for building web applications of all sizes. Flask is also known for its excellent documentation and community support, which makes it a great choice for developers who are just starting out with web development. Overall, Python with Flask is a powerful and flexible framework that is well-suited for developing small to medium-sized web applications, and it's a great choice for developers who are just starting out with web development.

Python with Flask for large-scale development

Flask is a popular framework for web application development. "micro" because its kernel is small and easy to extend, but it can be used for large-scale projects as well. Flask's flexibility makes it a good choice for large-scale applications because it allows developers to choose the components they need and to customize them to fit their specific requirements. Flask also has a large and active community, which means that there are many resources available for developers who are working on large-scale projects. Flask's documentation is also very comprehensive, which makes it easy for developers to get started with the framework and to find the information they need.

Flask is a good choice for large-scale applications for a few reasons. First, Flask is very modular, which means that developers can easily add or remove features as needed. This makes it easy to scale Flask applications as the application grows. Second, Flask has a large and active community, which means that there are many resources available for developers who are working on large-scale projects. Third, Flask's documentation is very comprehensive, which makes it easy for developers to get started with the framework and to find the information they need. Finally, Flask is a lightweight framework, which means that it has a small footprint and it is easy to deploy.

Here are some of the benefits of using Flask for large-scale applications:

- **Modularity:** Flask's modular design allows developers to easily add or remove features as needed. This makes it easy to scale Flask applications as the application grows.
- **Community support:** Flask has a large and active community, which means that there are many resources available for developers who are working on large-scale projects.
- **Documentation:** Flask's documentation is very comprehensive, which makes it easy for developers to get started with the framework and to find the information they need.
- **Lightweight:** Flask is a lightweight framework, which means that it has a small footprint and it is easy to deploy.

Here are some of the challenges of using Flask for large-scale applications:

- **Performance:** Flask is not as performant as some other frameworks, such as Django. This is because Flask is a lightweight framework and it does not have as many built-in features.
- **Security:** Flask is not as secure as some other frameworks, such as Django. This is because Flask does not have as many built-in security features.
- **Complexity:** Flask can be complex to use for large-scale applications. This is because Flask requires developers to manually configure many of the features that are built-in to other frameworks.

Overall, Python with Flask is a powerful and flexible framework that is well-suited for developing small to medium-sized web applications. However, it is important to be aware of the challenges of using Flask for large-scale applications before choosing it for your project.



Key Flask capabilities

- **Extensibility and integrations.** Flask's extensible architecture allows developers to add or remove features as needed. This makes it easy to integrate Flask with other Python libraries and frameworks, enabling developers to customize its functionality and create feature-rich web applications. This modularity is a key strength.
- **Transparent documentation.** Flask's documentation is published, allowing developers to quickly find the information they need to get started with the framework and to troubleshoot issues.
- **Custom implementation.** Each of the two customization and custom classes can be used for things like the request and response objects. The Flask class has many methods designed for subclassing. You can quickly add or customize behavior by subclassing Flask and using that subclass wherever you'd instantiate an application class.
- **Scaling considerations.** You can run scaling in such a way that if you double the number of servers you get about twice the performance. There is only one limiting factor regarding scaling in mind which is the use of threads in the process. This depends on context and it's hence a default scaling rather unlimited processes. Running in multiple processes is recommended instead of increasing the number of threads per process. Flask utilizes Google's thread locals to support these single-process.
- **Modular development.** Good for ways it will let you program can be written into a collection of utilities and shared among applications. Flask's modular design encourages developers to create reusable components, which can be shared among different projects. This helps to reduce development time and improve the overall efficiency of the development process. Flask's modular design also makes it easy to test individual components, which helps to ensure the quality of the application.

Real-world applications

Today, Python with Flask has become a popular choice among businesses for its simplicity, flexibility, versatility and easy-of-learning and use. Its minimalistic design and customizable nature make it adaptable, effective, and suitable for large-scale web development requirements in diverse industries and sectors.

Several prominent companies including Netflix, Reddit, LyftLinkedin, Pinterest and other leverage Python with Flask in their technology stacks for specific backend services or functionalities. Python Flask benefits big companies like Netflix and Reddit by providing a lightweight framework that can be easily customized to meet their specific needs, ensuring flexibility. Flask's modularity facilitates the addition of new functions within their infrastructure. This suggests that it can be part of a scalable architecture when combined with appropriate strategies and tools.

Author(s)

Rucha Kadam

Mark as completed